

ORDEM DE EXECUÇÃO/ENCERRAMENTO DE PROCESSOS GERADOS EM ORDEM PELO `FORK()`

Brenno Siqueira Ferreira Dias

IFCE

1. ABSTRACT

Hipótese: A ordem em que os processos filhos são criados influencia a ordem em que eles serão executados e terminarão. Consequentemente, ao utilizar `wait()`, o processo pai tenderá a receber os filhos na mesma ordem em que foram criados.

Para investigar essa hipótese, foram realizados experimentos em um sistema Linux(Mint) utilizando a linguagem C, onde foram criados 10 processos filhos por experimento. Foram realizadas três baterias de testes, cada uma contendo 500 experimentos: a primeira com tempos de espera variáveis e uma semente baseada no horário atual do sistema(`srand(time(NULL))`); a segunda com tempos de espera variáveis e uma semente fixa; e a terceira sem a utilização de `sleep()`, fazendo com que os processos executassem operações de duração iguais/parecidos. A ordem de término dos processos foi determinada por meio da chamada de sistema `wait()`, utilizando o PID retornado para identificar o processo que havia terminado. Os resultados demonstraram que a ordem de criação não garante a mesma ordem de término. Nos experimentos com tempos variáveis, a ordem de finalização foi bastante divergente da ordem de criação. Mesmo nos experimentos sem `sleep()`, foram identificados casos em que a ordem de término diferiu da ordem de criação, mesmo que mais raros. Portanto, os resultados indicam que a ordem de criação dos processos não estabelece uma ordem para sua finalização.

2. PALAVRAS-CHAVE

C, Fork, Wait, Waitpid, processos, ordem, finalização, Unix, escalonamento, sistemas operacionais

3. INTRODUÇÃO

Hipótese: A ordem em que os processos filhos são criados influencia a ordem em que eles serão executados e terminarão. Consequentemente, ao utilizar `wait()`, o processo pai tenderá a receber os filhos na mesma ordem em que foram criados.

Problema: Processos criados em programas reais criados pelo `fork` podem ter exatamente o mesmo código do pai ou

código diferente, ou seja, seu tempo de execução pode ser parecido para todos ou podem ter tempos diferente, maior ou menor. Então para verificarmos os dois casos precisamos criar processos em sequência e testar se os processos com o mesmo tempo de execução finalizam todos sempre na mesma ordem, e se o mesmo acontece quando eles têm tempos de execução diferente.

Realizamos testes com amostras de 10 processos por experimento e analisamos a ordem de término. Foram no total 3 baterias de testes: Tempo variado para cada processo com semente aleatória(ao atual da máquina); Tempo variado para cada processo com semente fixa; Tempo Fixo para cada processo;

4. METODOLOGIA

4.1. Ambiente

Sistema operacional Linux: Mint

Linguagem utilizada: C

Compilador: gcc

Processos criados: 10 por experimento com a chamada de sistema `fork()`

4.2. Implementação

`fork()`: para criação dos processos;

`wait()`: para esperar o término de um processo filho e obter seu PID;

`rand()`: para tempos aleatórios;

`sleep()`: para espera dos processos

Os processos filhos foram criados sequencialmente por meio de um laço de repetição. Para cada processo foram armazenados, em uma estrutura (`struct`), o número do experimento, o número do filho, o PID, a ordem de criação, o tempo de espera e a ordem de término.

Após a criação dos processos, o processo pai utilizou a chamada `wait()` para aguardar o término dos processos filhos, esse que retorna o PID do filho que encerrou, dessa forma, dentro de um laço `for()` foi possível determinar a ordem de encerramento dos filhos comparando-os com as informações dos filhos salvos na `struct`.

A footnote can be used to report funding sources and other potential conflicts of interest. Use Times 9-point type, single-spaced.

4.3. Configuracao experimentos

Foram realizadas três baterias de experimentos, cada uma contendo 500 execuções, com 10 processos filhos em cada execução:

- processos com tempos de execução variáveis e semente aleatória;
- processos com tempos de execução variáveis e semente fixa;
- processos sem `sleep()`, executando operações de duração parecidos.

5. RESULTADOS

Os resultados encontrados a partir das experimentacoes sao os seguintes:

5.1. Processos com tempo variável e semente aleatória

Foram realizados 500 experimentos, cada um contendo 10 processos filhos. Os processos receberam tempos de espera diferentes, determinados por valores aleatorios ligados a semente do tempo atual do computador.

Os resultados demonstraram uma grande variação na ordem de término dos processos. Em diversos experimentos, os processos finalizaram em ordem diferente daquela em que foram criados.

5.2. Processos com tempo variável e semente Fixa

Foram realizados 500 experimentos, cada um contendo 10 processos filhos. Os processos receberam tempos de espera diferentes, determinados por valores aleatorios ligados a semente fixa, que adotei para essa semente o valor 2580.

Os resultados demonstraram uma grande variação na ordem de término dos processos, assim como no experimento da semente aleatoria, com o processos finalizando em ordem diferente da ordem em que foram criados. No entanto, como a semente foi a mesma em cada rodada de experimentacao, os processos repetiram o comportamento para todas as rodadas de experimentos.

5.3. Processos com tempo semelhante

Foram realizados 500 experimentos, cada um contendo 10 processos filhos. Nesse teste os processo nao receberam nenhum tempo de sleep, apenas a linha de impressao que era comum a todos, mudando apenas o valor do filho e o PID, que seria individual.

Os resultados demonstraram uma pequena variacao na ordem de encerramento em comparacao com os outros experimentos. A variacao aconteciam em poucos filhos, e eram bem raros de acontecer, no entanto ainda existiam alteracoes, mesmo que o tempo de execucao deles fosse semelhante.

Cont.Núm - ordem termino	ordem termino									
filho	1	2	3	4	5	6	7	8	9	10
1	22	14	12	14	15	10	14	17	19	13
2	22	12	16	14	19	16	18	13	13	7
3	10	19	18	9	16	14	16	16	15	17
4	11	20	12	15	14	16	10	19	19	14
5	15	15	20	18	16	13	16	10	14	13
6	13	16	12	18	18	18	14	12	12	17
7	11	11	14	13	14	21	16	15	16	19
8	14	11	22	18	13	14	15	12	13	18
9	21	22	12	11	15	13	15	17	12	12
10	11	10	12	20	10	15	16	19	17	20
Total Resultado	150	150	150	150	150	150	150	150	150	150

Fig. 1. *

(a) Processos com tempo variável e semente aleatória

Cont.Núm - ordem termino	ordem termino									
filho	1	2	3	4	5	6	7	8	9	10
1			150							
2									150	
3										150
4				150						
5							150			
6					150					
7						150				
8	150									
9								150		
10		150								
Total Resultado	150	150	150	150	150	150	150	150	150	150

Fig. 2. *

(b) Processos com tempo variável e semente fixa

Cont.Núm - ordem termino	ordem termino									
filho	1	2	3	4	5	6	7	8	9	10
1	150									
2		150								
3			149					1		
4			1	149						
5				1	149					
6					1	148	1			
7						2	147			1
8								2	146	2
9									3	146
10										2
Total Resultado	150	150	150	150	150	150	150	150	150	150

Fig. 3. *

(c) Processos com tempo de execução semelhante, sem `sleep()`

6. CONCLUSAO

Os experimentos demonstraram que a ordem de criação dos processos por meio de `fork()` não garante que eles sejam finalizados na mesma ordem. A ordem de término resulta da interação entre o escalonamento do sistema operacional e as condições de execução de cada processo. Quando os processos possuem tempos de execução diferentes, as diferenças de duração têm forte influência na ordem de término, mas nao a ordem em que foram criados, pois mesmo quando os processos possuem tempos de execução parecidos, observamos casos em que a ordem de termino ficou diferente da ordem de criação.